

Exploring lightweight CNNs, ViTs, and graph networks for Chest X-Rays

Elizaveta Kuzovkova

Abstract—Abnormalities detection in chest X-rays (CXR) is a major challenge in automated medical diagnostics, partly due to complexity of overlapping critical visual patterns in screening images and low resolution of scans. Large pre-trained neural networks showed remarkable results for this task, but in various resource limited settings they are not sustainable. In this project we study multi-label classification of 14 thoracic findings from the ChestMNIST dataset on lightweight architectures, training all models from scratch at 64×64 input resolution. We adapt ideas from several branches of deep learning: convolution, attention networks, vision transformers, graph networks. And compare them under a shared pipeline to assess both classification performance and computational resources used. All models remain under one megabyte and roughly ten minutes or less convergence. The graph architecture with interaction fusion reaches the best macro AUC of 0.773, closely followed by the CBAM attention CNN, while transformers fall short. These results offer practical guidance on the accuracy-cost trade-off for CXR classification in resource constrained environment.

Index Terms—Medical Image Analysis, Convolutional Neural Networks, Vision Transformers, Graph Convolutional Networks, Attention Mechanism

I. INTRODUCTION

Chest X-ray (CXR) imaging is one of the most common and affordable screening tools for assessing lungs health. However, interpreting a massive amount of images is a heavy workload for radiologists, and the visual signs of many abnormalities are subtle and easily missed. Deep learning has shown that these scans can be read automatically, highlighting suspicious regions or even producing full multi-label predictions, which supports getting faster and more reliable diagnosis. This makes automated CXR classification a practical and clinically relevant problem, and a useful setting in which to compare modern neural architectures.

CXR may show several findings at once, so the problem is multi-label and strongly imbalanced, with some findings appearing in well under one percent of images. Much of the state-of-the-art addresses this with large pretrained models at high resolution, sometimes combined with label-relationship modeling [1], [2] or transformer designs tailored to CXR [3]. These approaches perform well but are heavy, and it is less clear how the same ideas behave when compute is limited and models must stay small, which is relevant to low-resource or on-device deployment.

In this work we study several families of deep learning models for multi-label classification on the ChestMNIST dataset [4], all trained from scratch at 64×64 resolution under a tight compute budget. We keep every model lightweight and

compare them under a single shared training and evaluation setup, so that architectural differences, rather than hyperparameters tuning, drive the results. Our contribution is to adapt ideas from convolutional attention, vision transformers, and graph neural networks, into a common small-scale configuration, and to report how each behaves in terms of accuracy and cost. We hope this comparison is useful to practitioners who need a starting point for CXR classification when resources are constrained.

The main contributions of this report are:

- a shared multi-label pipeline for ChestMNIST, with CLAHE preprocessing, weighted binary cross-entropy (BCE) for imbalance, and per-class threshold tuning, applied identically to six architectures;
- lightweight adaptations of four model families: a baseline CNN, a CBAM-attention CNN, a pure and a hybrid (IEViT) vision transformer, and two label-graph (GCN) variants;
- a set of ablations isolating the effect of contrast enhancement, class weighting, attention, and label co-occurrence graphs; robustness study under Gaussian corruption of input;
- a comparison of all models along both accuracy and computational cost, intended as practical guidance for resource-constrained settings.

The remainder of this report is structured as follows. Section II reviews related work, and Section III presents the processing pipeline, Section IV shows the data and features. The learning framework and the model architectures are detailed in Section V, and their evaluation is carried out in Section VI. Concluding remarks are given in Section VII.

II. RELATED WORK

This section provides an overview of the recent developments in deep learning for CXR analysis, grouped by implemented architectures. We focus on relevant and adopted to the project approaches.

CNNs continue to be widely used architectures for medical imaging analysis, demonstrating great performance even with lightweight architectures. Such light models were also tested on MedMNIST images: 1) MobileNetV3-Small and EfficientNetV2-Small achieved stable performance even on a variety of augmented data [5]; 2) SmallCNN showed the best macro ROC-AUC and post-calibration error [6]; 3) optimized via Particle Swarm Optimization (PSO) CNN accomplished superior accuracy over state-of-the-art methods [7]. But the last two works did not include multi-label CXR images

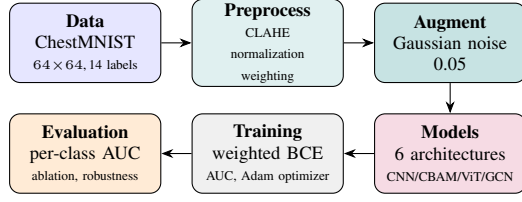


Fig. 1: Overview of ChestMNIST Processing Pipeline.

classification task. Attention mechanisms, including Squeeze-and-Excitation (SE) [8] and Convolutional Block Attention Module (CBAM) [9], has been explored for enhancing CNNs classification.

Transformer-based models, typically paired with CNN in a hybrid architecture, reached significant results in the area. Notable examples include MedViT [10], MedFormer [11], SwinMedNet [12], and IEViT [3] – approach adapted in this work as well.

In the recent years one more compelling branch emerged – GCN-based designs that enhance CXR classifiers by modelling co-occurrence relationships between pathologies [2] [13]. This cross-modal fusion enables the network to leverage global structural memory and comorbidity patterns to recalibrate its multi-label predictions.

This work is inspired by outlined methodological perspectives in testing the similar three domains of deep learning models: CNNs with attention, transformers, and graph-based networks.

III. PROCESSING PIPELINE

This project’s design consists of 6 main blocks, outlined in Figure 1. The additional block, which was not included in the schema, is a separate demo cases study that presents Grad-CAM heatmap for the explained finding and per-class probability bar chart.

The pipeline involves the following steps: 1) Data setup, exploration, and adjacency matrix creation; 2) Preprocessing and class balance adjustment; 3) Augmentation set creation; 4) Configuration and setup of 6 models to train; 5) Models training; 6) Performance evaluation, ablation study (turn off weighting), robustness to Gaussian noise check.

IV. SIGNALS AND FEATURES

A. Input Data

This study experiments on ChestMNIST dataset, which is a part of a large-scale collection of standardized medical images with classification labels. Due to limited computational resources, the images of $64 \times 64 \times 1$ size was used. The full scope of 14 classes was considered. Labels are (N,14) multi-hot vectors over the 14 lung findings, an all-zero vector denotes “no finding”.

The train/validation/test split was utilized from a predefined MedMNIST assignment, each fold consists of 78,468/11,219/22,433 samples respectively. The exploration of labels distribution showed a heavy imbalance of classes – about half (54%) of the train images attributed to “no finding”.

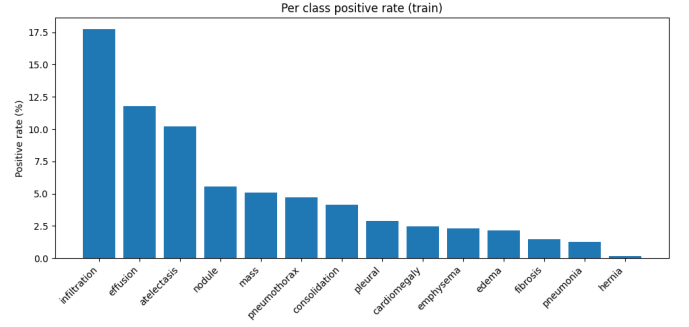


Fig. 2: Positives samples (%) per abnormality class.

As for each label finding, the most common class “infiltration” reaches only 17.7% , while the most rare class “hernia” was present in only 0.18% images. The full distribution is shown in Figure 2. Considering the number of categories per image, 27.5% pictures had one label, 12.7% of sample had 2 assignments, and the remaining part (5.7%) included 3 or more classes.

There is a minor inter-label correlation present with top pair reaching about 18% coefficient, such relationships between labels are used for graph head input. Two approaches to adjacency matrix construction were tested: a) the classical symmetric matrix based on conditional probabilities with threshold value for filtering the noisy edges [1] [2], which produced 28 edges; b) Pointwise Mutual Information (PMI) method – builds a continuous weighted matrix representing the normalized association between classes, generated 63 edges.

a) *Conditional-probability threshold.* From the training labels we first form the co-occurrence matrix $C = Y^T Y$, where C_{ij} counts the images in which findings i and j are both positive. Dividing each row by its diagonal gives the conditional probability that label j is present given label i ,

$$P_{ij} = \frac{C_{ij}}{C_{ii}} = P(j | i), \quad (1)$$

which we binarize with a threshold $\tau = 0.15$ to drop weak, noisy links. Since P_{ij} is directional, the resulting matrix is symmetrized by keeping an edge if either direction survives the threshold ($A = \max(A, A^T)$).

b) *PMI.* Instead of a hard cut, PMI measures how much more often two findings co-occur than expected if they were independent. Using the marginal rates p_i and the joint rate p_{ij} , we compute the normalized PMI (NPMI) scaling to $[-1, 1]$,

$$\text{NPMI}(i, j) = \frac{\log(p_{ij} / p_i p_j)}{-\log p_{ij}}, \quad (2)$$

and keep only positive associations ($A_{ij} = \max(\text{NPMI}(i, j), 0)$). This produces a continuous symmetric weight for every edge rather than a binary one, preserving the relative strength of each label relationship. PMI prevents highly frequent diseases from dominating co-occurrence links and captures subtle relationships for rare conditions that would otherwise look uncorrelated.

c) *Symmetric GCN normalization.* Both matrices are passed

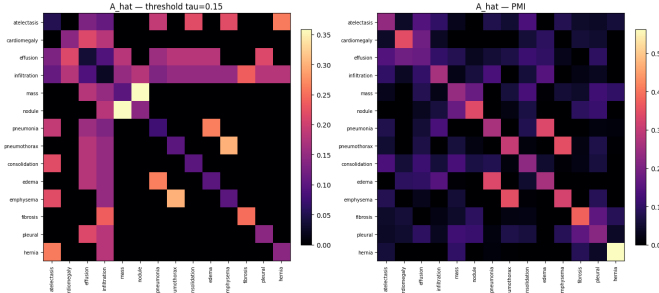


Fig. 3: Label Adjacency Matrices.

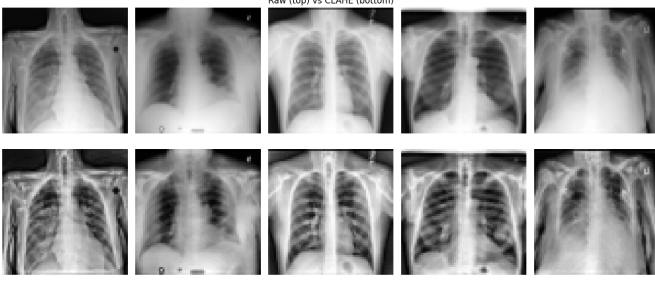


Fig. 4: CLAHE Processed Images.

through the same finalization step before use. Self-loops are first removed and re-added with a fixed weight $p = 0.25$, so each node retains a share of its own features ($A \leftarrow (1 - p)A + pI$); this keeps a label’s own embedding from being washed out by its neighbors. The matrix is then symmetrically normalized by its degree $D = \text{diag}(\sum_j A_{ij})$,

$$\hat{A} = D^{-1/2} A D^{-1/2}, \quad (3)$$

which prevents high degree findings from dominating the message passing and stabilizes the GCN layers. The result $\hat{A}$ is a fixed matrix shared across GCN variants. The resulting matrices visualization is in Fig. 3.

B. Preprocessing

CLAHE. The first step included Contrast Limited Adaptive Histogram Equalization (CLAHE) algorithm application – widely used image contrast improvement technique – with clipLimit=2.0, tileGridSize=8×8 parameters. The local enhancement of contrast helps to define tissue segmentation and highlight details essential for disease detection, especially on low-quality scans. The examples of images after CLAHE application are shown in Fig. 4

Normalization. Pixel values were scaled to $[0, 1]$ and standardized using statistics computed on the training split only. The same fixed mean and std are then applied to the validation and test splits, so no information about the held-out data influences preprocessing. Z-score normalization was preferred over plain $[0, 1]$ scaling because it centers the activations and tends to stabilize and speed up convergence.

Class imbalance. The loss function was reweighted to counter the extreme label imbalance. For each category we set a positive weight (negatives over positives), computed on the

training split, which up-weights the rare positive examples. We apply a square-root softening and clip the result to the range $[1, 15]$, giving the positive weight

$$w_c = \text{clip}\left(\sqrt{N_c^- / N_c^+}, 1, 15\right), \quad (4)$$

which is then used inside the weighted binary cross-entropy. The raw inverse-frequency weights reach several hundred for the rarest findings, which destabilized training and overinflated the loss on a few classes, as was tested in some runs. The square root compresses this range while preserving the ordering, and the cap bounds the most extreme values. Resampling or duplicating images was deliberately avoided: in a multi-label task it distorts the label co-occurrence structure and can cause leakage. Handling imbalance purely in the loss keeps the splits untouched and isolates the correction to training, while validation and test sets reflect the true class distribution.

C. Augmentation

The second training dataset was created with slight augmentation of normalized images. We opted for Gaussian noise addition $\sigma = 0.05$ to not disturb anatomical validity of CXR via flips or rotation. We also explore models performance degradation on the range of $[0, 0.5]$ Gaussian sigmas.

V. LEARNING FRAMEWORK

A. Training Configuration

All models were trained from scratch, without any pre-training. Main parameters and setup choices were shared across architectures. Each network outputs 14 raw logits and is trained with a weighted binary cross-entropy loss. We chose Adam optimizer for its adaptive per-parameter learning rate and robustness to noisy gradients, with a base learning rate of 0.001 for CNNs and a lower 0.0003 for transformer and graph variants, which are more sensitive to large updates. We use a batch size of 128, balancing gradient stability against memory on the Kaggle GPU, and train for up to 50 epochs. Two callbacks regulate training: ReduceLROnPlateau halves the learning rate when the validation AUC stops improving for ten epochs (down to a floor of 0.000001), allowing finer updates near convergence, while EarlyStopping (patience 15) restores the best weights and prevents overfitting.

B. Models

Baseline CNN. The minimal evaluated model we tested was simple 3 layer CNN with increasing filter sizes (32, 64, 128), BatchNormalization, ReLU activation, and MaxPooling layer that reduces the spatial dimensions and improves computational efficiency. The resulting feature maps are reduced by a global average pooling layer and passed through a dense layer of 256 neurons with ReLU activation, followed by a dropout to limit overfitting. The final dense layer has 14 neurons with a linear activation, producing one logit per finding for the multi-label task. The full architecture and per-layer parameters are reported in Tab. 1.

TABLE 1: M1 - Baseline CNN Architecture.

Layer (Type)	Output Shape	Param #
Input	(None, 64, 64, 1)	0
Conv2D 32 + BN + ReLU	(None, 64, 64, 32)	448
MaxPooling2D	(None, 32, 32, 32)	0
Conv2D 64 + BN + ReLU	(None, 32, 32, 64)	18,752
MaxPooling2D	(None, 16, 16, 64)	0
Conv2D 128 + BN + ReLU	(None, 16, 16, 128)	74,368
MaxPooling2D	(None, 8, 8, 128)	0
GlobalAvgPool2D	(None, 128)	0
Dense 256, ReLU	(None, 256)	33,024
Dropout 0.3	(None, 256)	0
Dense 14 (linear)	(None, 14)	3,598
Total params	130,190 (0.50 MB)	
Trainable	129,742	
Non-trainable	448	

TABLE 2: M2 - CNN + CBAM Attention Architecture.

Layer (Type)	Output Shape	Param #
Input	(None, 64, 64, 1)	0
Conv2D 32 + BN + ReLU	(None, 64, 64, 32)	448
CBAM (channel + spatial)	(None, 64, 64, 32)	261
MaxPool + Dropout 0.2	(None, 32, 32, 32)	0
Conv2D 64 + BN + ReLU	(None, 32, 32, 64)	18,752
CBAM (channel + spatial)	(None, 32, 32, 64)	679
MaxPool + Dropout 0.2	(None, 16, 16, 64)	0
Conv2D 128 + BN + ReLU	(None, 16, 16, 128)	74,368
CBAM (channel + spatial)	(None, 16, 16, 128)	2,283
GlobalAvgPool2D	(None, 128)	0
Dense 256, ReLU	(None, 256)	33,024
Dropout 0.4	(None, 256)	0
Dense 14 (linear)	(None, 14)	3,598
Total params	133,413 (0.51 MB)	
Trainable	132,965	
Non-trainable	448	

CNN with CBAM attention. This model keeps the same three-layers convolutional backbone but inserts a Convolutional Block Attention Module (CBAM) after each convolutional layer, so that the network can adaptively focus on the most informative features and regions. Each CBAM applies channel attention using average and max pooling, and then spatial attention based on pooled channel descriptors, which highlights relevant regions. This combination is well suited to CXR, where findings are often localized to specific areas of the lung. As in the baseline, a global average pooling layer, a dense layer of 256 neurons, and a dropout layer precede the final 14-logit output. Notably, CBAM adds only about 3,200 parameters over the baseline (Tab. 2), making it an attractive lightweight option.

ViT-lite. This model is a pure Vision Transformer trained from scratch, included as a transformer baseline. The input image is first split into non-overlapping 8×8 patches by a strided convolution, producing 64 patch tokens of dimension 64; a learnable class token is prepended and learnable positional embeddings are added so that spatial order is retained. The token sequence is then processed by four transformer encoder blocks, each applying multi-head self-attention (4 heads) followed by a two-layer GELU MLP, with layer normalization and residual connections. The final class token is passed to a dense layer of 14 logits. A full specification is given in Tab. 3.

IEViT-lite. This model (Tab. 4) is an enhanced ViT that improves the pure ViT with convolutional features, following the IEViT architecture of Okolo et al. [3], which was proposed

TABLE 3: M3a - ViT-lite Architecture.

Layer (Type)	Output Shape	Param #
Input	(None, 64, 64, 1)	0
PatchConv 64, 8×8 , s8	(None, 8, 8, 64)	4,160
Reshape (tokens)	(None, 64, 64)	0
ClassToken	(None, 65, 64)	64
AddPosEmbed	(None, 65, 64)	4,160
Transformer enc. $\times 4$	(None, 65, 64)	133,888
LayerNorm	(None, 65, 64)	128
Extract [CLS]	(None, 64)	0
Dense 14 (linear)	(None, 14)	910
Total params	143,310 (0.55 MB)	
Trainable	143,310	
Non-trainable	0	

TABLE 4: M3b - IEViT-lite (CNN + Transformer) Architecture.

Layer (Type)	Output Shape	Param #
Input	(None, 64, 64, 1)	0
CNN token branch	(None, 1, 64)	23,360
PatchConv 64, 8×8 , s8	(None, 64, 64)	4,160
ClassToken + PosEmbed	(None, 65, 64)	4,224
Transformer enc. $\times 3$ (inject)	(None, 65, 64)	100,416
LayerNorm	(None, 65, 64)	128
Extract [CLS]	(None, 64)	0
Dense 14 (linear)	(None, 14)	910
Total params	133,198 (0.51 MB)	
Trainable	133,006	
Non-trainable	192	

specifically for CXR classification. The key idea taken from that work is that a shared CNN-extracted feature is injected into *every* transformer block rather than used only once. In this project a lightweight CNN branch (two convolutional blocks followed by global average pooling and a dense layer) produces a single global token g , which is concatenated to the patch-token sequence at the input of each encoder block and then removed before the next block, so the sequence length stays fixed. We keep the design deliberately small for the 64×64 grayscale setting and the limited computational resources. Thus, the comparison of ViT-lite and IEViT-lite presents a clean ablation study to assess the impact of CNN-feature injection.

GCN-lite with dot-product fusion. This model adds a label-graph branch to the CNN backbone, following the multi-label GCN approach of Chen et al. [1], [2] (ML-GCN/CheXGCN), where the relationships between co-occurring classes are modeled explicitly rather than predicted independently. The image branch reuses the CBAM model specification and produces a feature vector $F \in \mathbb{R}^{256}$ per image. In parallel, a label branch takes 14 learnable label-node embeddings and propagates them through two graph convolutional layers over a fixed adjacency matrix $\hat{A}$. This produces a set of class-specific weight vectors $W \in \mathbb{R}^{14 \times 256}$ that encode each finding together with its graph neighbors. The two branches are combined by a dot-product, so that the score for class n is the alignment between the image feature and that class's graph-informed weight,

$$\text{logit}_n = F \cdot W_n^\top, \quad (5)$$

which is exactly the ML-GCN fusion. The original works rely on a large-scale pre-trained models, GloVe word embeddings, and high-resolution inputs. We adopt a lightweight version

TABLE 5: M4a - GCN-lite, Dot Fusion Architecture.

Layer (Type)	Output Shape	Param #
Input	(None, 64, 64, 1)	0
<i>Image branch (CBAM backbone)</i>		
Conv32 + BN + ReLU + CBAM	(None, 64, 64, 32)	709
MaxPool + Dropout	(None, 32, 32, 32)	0
Conv64 + BN + ReLU + CBAM	(None, 32, 32, 64)	19,431
MaxPool + Dropout	(None, 16, 16, 64)	0
Conv128 + BN + ReLU + CBAM	(None, 16, 16, 128)	76,651
GAP + Dense 256 $\rightarrow F$	(None, 256)	33,024
<i>Label branch (graph)</i>		
LabelEmbeddings 14 $\times$ 64	(None, 14, 64)	896
GCN 64 $\rightarrow$ 128	(None, 14, 128)	8,192
GCN 128 $\rightarrow$ 256 $\rightarrow W$	(None, 14, 256)	32,768
DotFusion $F \cdot W^T$	(None, 14)	0
Total params	171,671 (0.65 MB)	
Trainable	171,223	
Non-trainable	448	

trained from scratch (CBAM, learnable label embeddings, 64 $\times$ 64 input) suited to the limited computational budget. The full specification is given in Tab. 5.

GCN-lite with interaction fusion. This variant keeps the same image and label branches but replaces the dot-product with a richer bilinear interaction, inspired by the feature-interaction fusion of GCF-Net [13]. The image feature F and each class weight W_n are first projected into a shared m -dimensional space, combined element-wise, and pooled into one score per class,

$$\text{logit}_n = \sum_{k=1}^m (F P_F)_k \odot (W_n P_W)_k, \quad (6)$$

where $P_F, P_W \in \mathbb{R}^{256 \times m}$ are learnable projections and $\odot$ is the Hadamard product. This lets the model capture multiplicative interactions between image features and label structure that a dot-product cannot. Both GCN models are evaluated with the two adjacency matrices described in Sec. IV, so that the effect of the graph construction can be compared independently of the fusion strategy. This gives a three step ablation strategy: from the CNN with CBAM (M2) to dot-product fusion (M4a) to interaction fusion (M4b), isolating the contribution of label co-occurrence and of the fusion mechanism in turn.

C. Evaluation Metrics

Model performance was evaluated on two aspects: classification quality and computational efficiency. Since the task is multi-label, all classification metrics are first computed per finding and then averaged over the $C = 14$ classes.

Per-class quality (threshold-free): The primary metric is the area under the ROC curve (AUC-ROC), which measures how well a finding’s predicted probability separates positive from negative images across all possible decision thresholds. It is threshold-independent and robust to class imbalance, which makes it the standard metric for the task. We report its macro average,

$$\text{AUC}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \text{AUC}_c. \quad (7)$$

Mean average precision (mAP): The average precision summarizes the area under the precision-recall curve for each

class, and mAP averages it over all classes,

$$\text{AP}_c = \sum_k (R_k - R_{k-1}) P_k, \quad \text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c, \quad (8)$$

where P_k and R_k are the precision and recall at the k -th threshold. Because mAP emphasizes the positive (rare) class, it is more informative than AUC when positives are scarce, as is the case for several labels here.

Per-class quality (thresholded): To obtain predictions, the probabilities are compared against a per-class decision threshold. Precision and recall are derived from the true positives (TP), false positives (FP), and false negatives (FN) of each class,

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (9)$$

and combined into the F_1 score, their harmonic mean,

$$F_1 = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (10)$$

Since a fixed cutoff can be a poor measure under imbalance condition, the threshold of each class is tuned on the validation split to maximize its F_1 , and these thresholds are then applied once to the test split. We also report the number of positive test instances per class (the support) to show per-class scores for rare findings.

Aggregate F_1 : Two averaging approaches are used. The macro- F_1 is the unweighted mean of per-class F_1 values, which gives every finding equal importance regardless of frequency,

$$F_1^{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F_{1,c}. \quad (11)$$

The micro- F_1 instead pools the TP, FP, and FN counts across all classes before computing a single score, so frequent findings contribute proportionally more,

$$F_1^{\text{micro}} = \frac{2 \sum_c \text{TP}_c}{2 \sum_c \text{TP}_c + \sum_c \text{FP}_c + \sum_c \text{FN}_c}. \quad (12)$$

Reporting both separates per-class behavior (macro) from overall prediction quality dominated by common findings (micro).

Computational efficiency: To assess model complexity we report the total number of trainable parameters, the model size in megabytes, and the mean inference time per image, averaged over a batch on the Kaggle GPU. We also record the training time of each model. Together these capture the accuracy-cost trade-off that is central to deployment in limited resource settings.

VI. RESULTS

In this section we go over models performance results, then examine resource usage per each model. Afterwards we report the demonstrated difference in selected models performance with turned off class weights correction, various Gaussian noise sigmas, and show some prediction examples

TABLE 6: Comparison of classification performance. The best result is shown in **bold**.

Model	Mac AUC	mAP	Mic F1	Mac F1
GCN inter (PMI)	0.7725	0.1596	0.2796	0.2191
CNN + CBAM	0.7712	0.1523	0.2909	0.2142
GCN inter (Thr)	0.7669	0.1494	0.2776	0.2086
GCN dot (PMI)	0.7660	0.1465	0.2856	0.2037
GCN dot (Thr)	0.7659	0.1487	0.2694	0.2054
Baseline CNN	0.7616	0.1431	0.2753	0.1987
IEViT-lite	0.7375	0.1288	0.2603	0.1834
ViT-lite	0.7137	0.1215	0.2400	0.1746

using Gradient-weighted Class Activation Mapping (Grad-CAM) technique to understand which image regions were the most influential for the model’s decision about CXR labels.

A. Performance Results

The classification performance of all models, trained on clean data, is reported in Tab. 6, ranked by macro AUC-ROC. For all models macro AUC reaches between 0.714 and 0.773, which is close to the results reported in studies with pre-trained large models. The absolute values of the precision-based metrics (mAP around 0.12-0.16, macro F_1 around 0.17-0.22) are low, as expected for a 14 label problem with strong class imbalance, where most findings are positive in only a small fraction of images. We rely primarily on macro AUC, which is threshold-free and imbalance-robust, and read the remaining metrics as complementary.

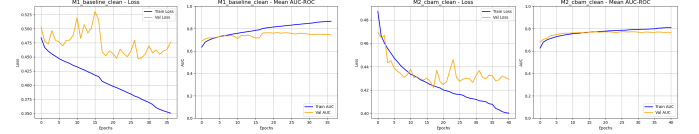
Overall, the graph-based model with interaction fusion (PMI adjacency) achieves the best macro AUC (0.7725), mAP and macro F_1 , closely followed by the attention CNN (CNN + CBAM, 0.7712), which reaches the best micro F_1 . Examining the per-class results of the best model in Tab. 7 reveals a wide spread across findings that the macro average does not show. The model discriminates best on several abnormalities: cardiomegaly (0.873), edema (0.858), effusion (0.837) and pneumothorax (0.799) all exceed 0.80 AUC, and hernia reaches the single highest score (0.901) despite having only 42 positive test cases. In the second best performing CNN + CBAM attention model hernia detection is lower (0.8787). When class weighting is removed (see Section VI-C), CBAM loses 0.017 mean rare class AUC while the graph model stays flat, which points to the beneficial role of co-occurrence prior for infrequent diseases.

The main observations are as follows:

- **Attention and label graph priors both help.** The CBAM CNN and the GCN variants improve over the baseline CNN (0.7616), confirming that both spatial-channel attention and label co-occurrence add useful (but modest in this case) discriminative power over a simple convolutional model.
- **Transformers trail the convolutional models.** The pure ViT is last (0.7137), the hybrid IEViT-lite recovers about 0.024 AUC over it (0.7375) by injecting a shared CNN feature into every transformer block. Both still fall below the CNNs, which is consistent with the known difficulty

TABLE 7: GCN interaction PMI: per-class classification performance.

Label	AUC	AUPRC	Prec.	Rec.	F1
Atelectasis	0.754	0.250	0.271	0.397	0.322
Cardiomegaly	0.873	0.240	0.246	0.366	0.294
Effusion	0.837	0.416	0.378	0.611	0.467
Infiltration	0.674	0.307	0.285	0.528	0.370
Mass	0.726	0.129	0.150	0.333	0.207
Nodule	0.656	0.121	0.156	0.198	0.174
Pneumonia	0.681	0.024	0.033	0.178	0.055
Pneumothorax	0.799	0.186	0.232	0.315	0.267
Consolidation	0.781	0.124	0.126	0.525	0.203
Edema	0.858	0.100	0.131	0.213	0.162
Emphysema	0.798	0.097	0.114	0.322	0.169
Fibrosis	0.751	0.046	0.055	0.232	0.089
Pleural	0.726	0.084	0.088	0.431	0.146
Hernia	0.901	0.112	0.143	0.143	0.143



(a) M1 - Baseline CNN training curves (b) M2 - CNN+CBAM training curves

Fig. 5: Training curves for the CNN models.

of training vision transformers from scratch on small low-resolution datasets.

- **Convergence and early stopping varied.** All models were trained for up to 50 epochs with early stopping on validation macro AUC. The convolutional and transformer models met the stopping criterion early, while all four graph variants ran to the full limit. In every case validation AUC plateaued well before training stopped (typically by epoch 10-15), so the extra epochs mainly reflect slow improving graph training rather than continued gains for the CNN and ViT models. Consistent with the calibration behavior noted earlier, the graph models showed the most stable validation loss. The baseline CNN and the transformers demonstrated the noisiest validation loss curves despite flat validation AUC.

B. Resource Usage

Beyond accuracy, Tab. 8 reports the computational cost of each model in terms of parameter count, size, training time, and mean inference time per image. All models are deliberately lightweight: every variant fits under one megabyte, spanning only 130k to 204k parameters, which keeps them suitable for resource-constrained deployment. The key trade-offs are the following:

- **The baseline CNN is the cheapest.** At 130k parameters and 294s it is the fastest, while still reaching a competitive 0.7616 macro AUC, making it a strong low cost reference point.
- **CBAM adds attention almost for free.** The CBAM CNN adds only about 3k parameters over the baseline, while it gains nearly a point of macro AUC, offering the

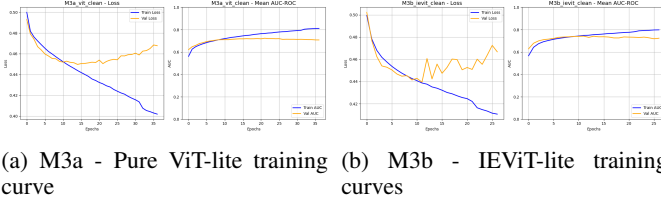


Fig. 6: Training curves for the ViT models.

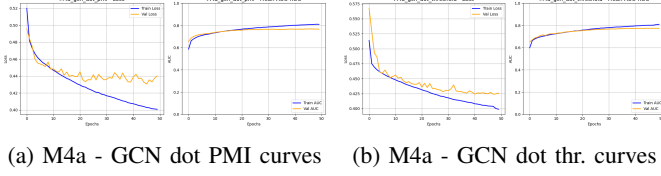


Fig. 7: Training curves for the GCN with dot product models.

best accuracy per parameter of all models and the fastest inference time.

- **The graph models are the most accurate but the most expensive.** The GCN variants carry the largest parameter counts (up to 204k) and the longest training times because they combine the full CBAM image with the label graph branches.
- **Transformers are cheap to train but not competitive.** The ViT and IEViT models train quickly but their lower accuracy means the CNN and graph models dominate in the trade-off between cost-accuracy at this scale.

TABLE 8: Comparison of computational efficiency.

Model	Params	Size(MB)	Time(s)	Inf.(ms)
GCN inter (PMI)	204k	0.78	641	0.20
CNN+CBAM	133k	0.51	488	0.17
GCN inter (Thr)	204k	0.78	639	0.19
GCN dot (PMI)	172k	0.65	615	0.19
GCN dot (Thr)	172k	0.65	631	0.19
Base CNN	130k	0.50	294	0.18
IEViT-lite	133k	0.51	319	0.18
ViT-lite	143k	0.55	308	0.22

C. Class Weighting Off

To isolate the effect of the class weighting, we retrained the two strongest models at the time of the ablation – CBAM and the GCN with dot product fusion (PMI) – with the positive weights disabled. At the macro level the effect is small and inconsistent: CBAM loses 0.0045 AUC while the GCN slightly gains (0.7660 $\rightarrow$ 0.7696). The relevant effect is on the rare findings, where ablation costs CBAM -0.017 mean AUC (hernia drops 0.879 $\rightarrow$ 0.832), but leaves the GCN essentially unchanged (+0.002). This may indicate that the co-occurrence graph acts as its own imbalance buffer and the GCN depends less on the loss correction. It should be noted that without weighting the tuned thresholds collapse to the bottom of the grid and hernia’s F_1 falls to zero, so the flat macro AUC is partly an artifact of AUC being threshold-free.

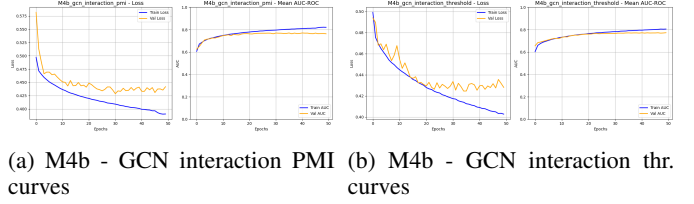


Fig. 8: Training curves for the GCN with interaction fusion models.

D. Robustness Analysis

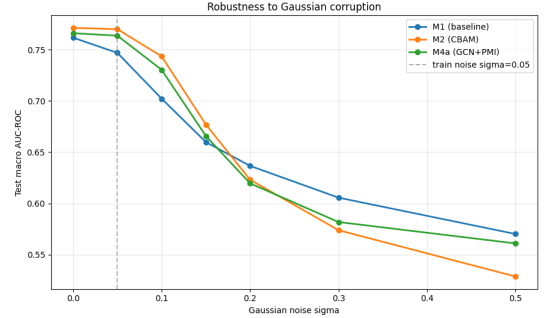


Fig. 9: Robustness to Gaussian noise slopes.

Robustness to input augmentation was evaluated by applying increasing σ Gaussian noise to the test images and measuring the macro AUC of the frozen models, comparing the baseline CNN, CBAM, and the GCN (dot product, PMI). As shown in Fig. 9, the ordering reverses under corruption: the baseline degrades the least (-0.19), the GCN is intermediate (-0.21), and CBAM degrades the most (-0.24). This rejects the initial hypothesis that attention and graph priors would be more stable. One possible explanation is that attention concentrates the model’s response on specific regions and channels, which improves clean accuracy but makes it more sensitive when those regions are corrupted, whereas the plain CNN bases its predictions on a more uniformly distributed set of image features.

E. Grad-CAM Examples

Finally, we inspect the model’s decisions using Gradient-weighted Class Activation Mapping (Grad-CAM) [14], applied to the CBAM model. Grad-CAM produces a heatmap over the input image highlighting the regions that most influenced the predictions. Fig. 10 and Fig. 11 show two representative cases: a correctly predicted multi-label CXR and a case with a missed finding.

VII. CONCLUDING REMARKS

In this study we compared six lightweight deep learning models for multi-label CXR classification task. Most networks demonstrated a very similar performance on the main metric, but their per-class detection, specifically of rare labels, varied. The category co-occurrence graph addition improved rare class ranking and reduced reliance on label imbalance reweighting, even under modest correlation between findings.

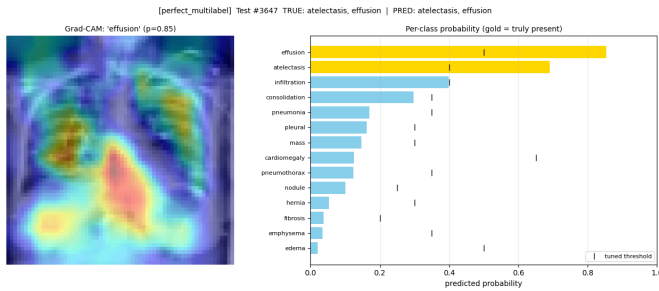


Fig. 10: Grad-CAM: Correct multi-label finding.

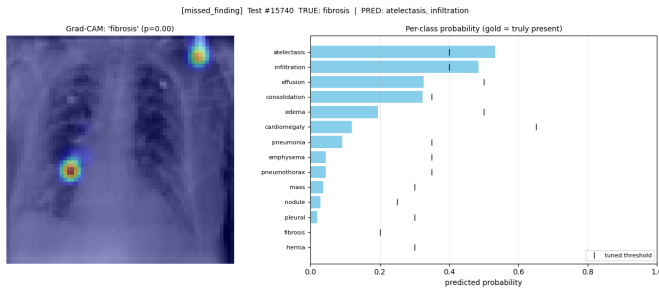


Fig. 11: Grad-CAM: Missed finding.

Several directions could be explored further:

- The graph construction could be made learnable rather than fixed adjacency matrix input.
- The robustness study may be expanded to other types of corruption: rotations, blur, crop, random shear, etc.
- Evaluating the same architectures on higher resolution images could prove to be advantageous, especially for ViTs performance that generally benefits from richer spatial detail.

From a personal perspective, this project gave me valuable hands-on experience in implementing a variety of deep learning methods on medical data. In my literature review research I found hybrid graph approaches to be particularly impressive, and adapting them to limited computational resource was challenging. Moreover, finding a stable enough solution for validation loss convergence proved to be difficult at first due to heavy class imbalance in the dataset, dealing with which is a beneficial training by itself.

REFERENCES

- [1] Z.-M. Chen, X.-S. Wei, P. Wang, and Y. Guo, "Multi-label image recognition with graph convolutional networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5177–5186, 2019.
- [2] B. Chen, J. Li, G. Lu, H. Yu, and D. Zhang, "Label co-occurrence learning with graph convolutional networks for multi-label chest x-ray image classification," *IEEE journal of biomedical and health informatics*, vol. 24, no. 8, pp. 2292–2302, 2020.
- [3] G. I. Okolo, S. Katsigiannis, and N. Ramzan, "Ievit: An enhanced vision transformer architecture for chest x-ray image classification," *Computer Methods and Programs in Biomedicine*, vol. 226, p. 107141, 2022.
- [4] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "Medmnist v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification," *Scientific data*, vol. 10, no. 1, p. 41, 2023.
- [5] A. R. C. Putra and S. Devella, "Comparison of mobilenetv3-small and efficientnetv2-small for low-resolution x-ray image classification," *Journal of Innovation and Technology Polbeng Series on Informatics (INOVTEK Polbeng-Seri Informatika)*, vol. 11, no. 1, pp. 215–225, 2026.
- [6] S. Lu, X. Chang, and T. Zou, "Uncertainty-aware medical vision-language classification on a lightweight medmnist-compatible biomedical patch benchmark," *Journal of Technology Informatics and Engineering*, vol. 5, no. 2, pp. 01–19, 2026.
- [7] Z. Al-Milaji and H. Yousif, "Lightweight deep learning model optimization for medical image analysis," *International Journal of Imaging Systems and Technology*, vol. 34, no. 5, p. e23173, 2024.
- [8] H. Fang, H. Xu, X. Sun, X. Zhang, and T. T. Toe, "Resnet models with se attention for lung diseases detection," in *2024 IEEE 6th International Conference on Power, Intelligent Computing and Systems (ICPICS)*, pp. 462–468, 2024.
- [9] I. Das, M. A. Sheakh, S. Abdulla, M. S. Tahosin, M. M. Hassan, S. Zaman, and A. Shukla, "Improving medical x-ray imaging diagnosis with attention mechanisms and robust transfer learning techniques," *IEEE Access*, 2025.
- [10] O. N. Manzari, H. Ahmadabadi, H. Kashiani, S. B. Shokouhi, and A. Ayatollahi, "Medvit: a robust vision transformer for generalized medical image classification," *Computers in biology and medicine*, vol. 157, p. 106791, 2023.
- [11] C. Simionescu, "Medformer: A multitask multimodal foundational model for medical imaging," *Procedia Computer Science*, vol. 270, pp. 446–455, 2025.
- [12] M. M. Haque, A. F. Ashrafi, S. Akter, and S. Mahamad, "Swinmednet: A hierarchical transformer for cross-modal medical image classification," *Available at SSRN 5273371*.
- [13] J. Sun, J. Hu, X. Wu, Z. Xu, Y. Wang, and Y. Zhang, "Multi-label chest x-ray image classification based on graph convolutional networks and multi-modal fusion," *Biomedical Signal Processing and Control*, vol. 119, p. 109930, 2026.
- [14] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," *International Journal of Computer Vision*, vol. 128, no. 2, pp. 336–359, 2019.